UNIVERSITY OF MICHIGAN

AE567

Statistical Inference, Estimation, and Learning

Project 2:

Part 2: Bayesian Inference for Nonlinear Models

Deniz Dost

UM ID: 82337583

Nov. 9, 2021
Deniz Dost, 82337583

1. Delayed Rejection Adaptive Metropolis

In this section of the project, four algorithms are implemented, Standard Metropolis Hastings (MH), Adaptive Metropolis Hastings (AM), Delayed Rejection Metropolis Hastings (DR), and Delayed Rejection Adaptive Metropolis Hastings (DRAM). These are used on the simple banana shaped distribution with two dimensional posterior $\pi(x_1, x_2)$ as:

$$(x_1, x_2 + (x_1^2 + 1)) \sim N\left(\mu, \Sigma\right) = N\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}\right)$$

This banana function is created by

$$logpdf = log\left(\frac{1}{(2\pi)^{\frac{n}{2}}|\Sigma|^{\frac{1}{2}}} exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)\right)$$

Then, the posterior function for this part looks like as shown in Figure 1.

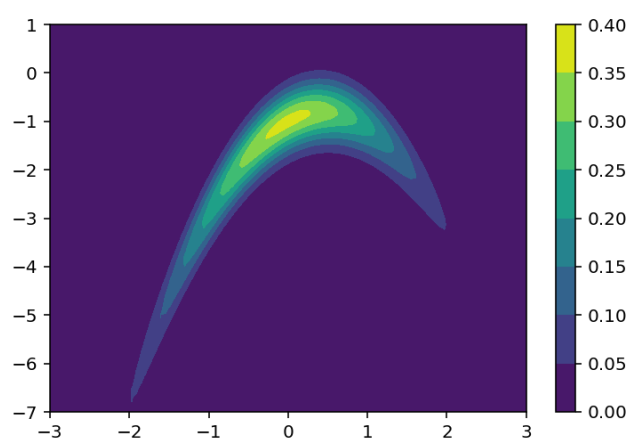

Figure 1: Banana logpdf function

Then, Laplace Approximation is used to optimize the posterior and compute the approximate hessian. So, a MAP point and a covariance matrix is obtained from this posterior to be used in the MCMC algorithms as the initialization point and covariance. These values are obtained as follows:

$$MAP = [-7.67x10^{-6}, -1]$$

$$\Sigma_{Laplace} = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$$

And MAP can be seen on the banana posterior function as shown below in Figure 2.

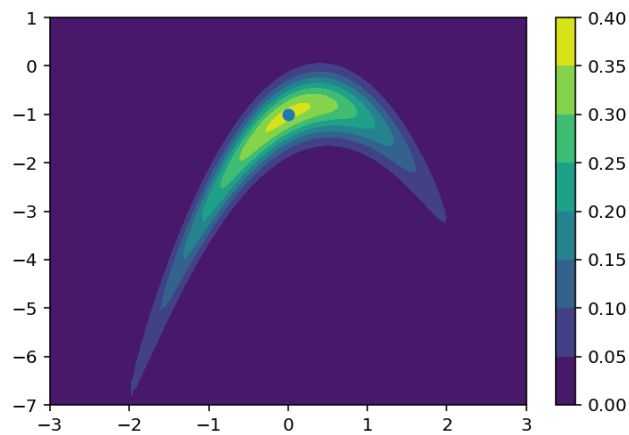

Figure 2: MAP on Banana Logpdf

1.1. Metropolis Hastings MCMC (MH)

MARKOV CHAIN MONTE CARLONUN BES SARTI

In the MH algorithm, first, an initial value $x^{(0)}$ is set to the state. Then, a proposed state is sampled from a distribution $y \sim (q(y|x^{(k)}))$. An acceptance probability for that proposal is computed as shown below.

$$a_1(x,y) = min\left(1, \frac{f_X(y)q_1(x|y)}{f_X(x)q_1(y|x)}\right)$$

Considering the Gaussian distribution, q_1's are equal to each other, so they cancel.

Finally, the proposed state is accepted or rejected according to that probability

$$x^{(k+1)} = \begin{cases} y\ , & w/\ probability \quad a(x^{(k)}, y) \\ x^{(k)}\ , & otherwise \end{cases}$$

The 1D and 2D marginal plots for MH is shown below in Figure 3.

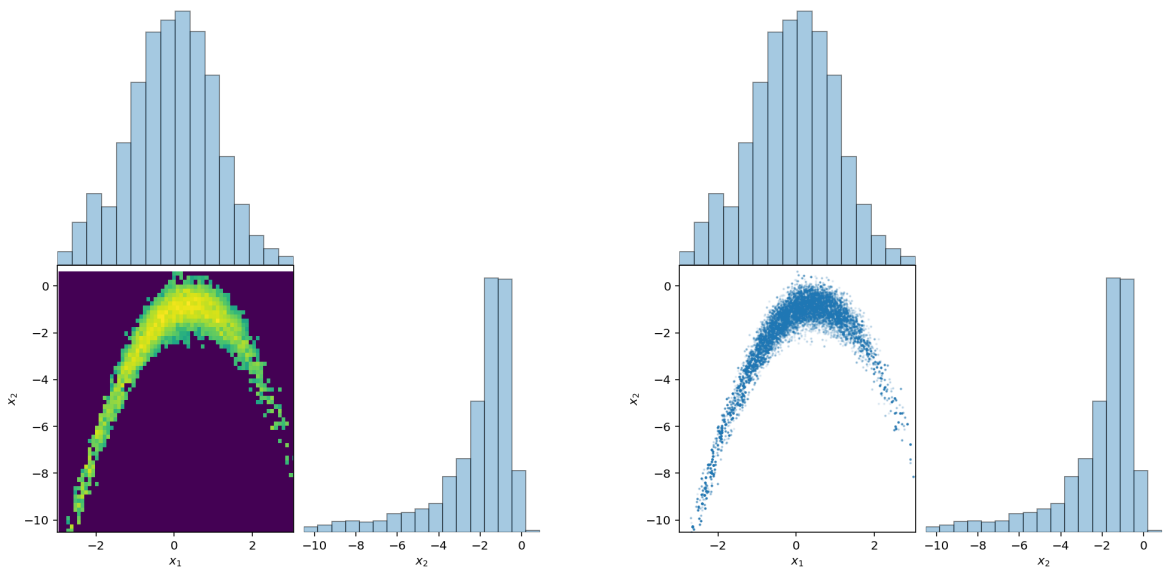

Figure 3: 1D and 2D Marginal Plots for MH

The autocorrelation plot is shown below in Figure 4.

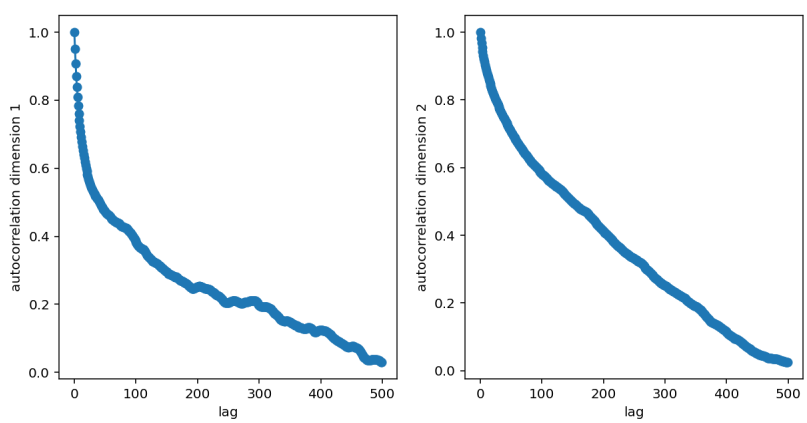

Figure 4: Autocorrelation Plots for MH

The integrated autocorrelation can be calculated from the autocorrelation values as shown below:

$$IAC = 1 + 2\sum_{L=1}^{\infty} corr(f(X_0), f(X_l))$$

The integrated autocorrelation value is 254.85 for the first dimension and 361.88 for the second dimension.

The acceptance ratio is 0.26.

Visual inspection of mixing is shown below in Figure 5.

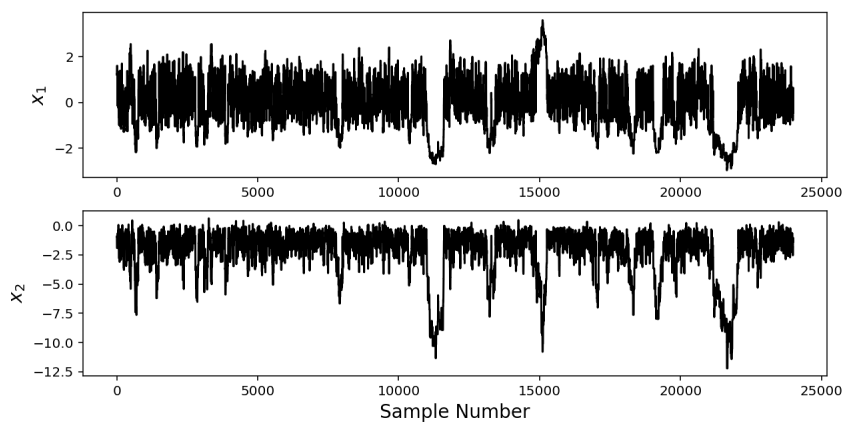

Figure 5: Visual Inspection of Mixing for MH

1.2. Adaptive Metropolis Hastings MCMC (AM)

For the AM, the proposal function is updated from the previous time step knowledge by updating the covariance as governed by the following formula

$$S_{k+1} = \frac{k-1}{k} S_k + \frac{s_d}{k}\left[\xi I + k\,\bar{x}^{k-1}\bar{x^{k-1}}^T - (k+1)\bar{x}^k\bar{x}^{k^T} + x^{(k)}x^{(k)^T}\right], \quad where \quad s_d = \frac{2.4^2}{d}$$

where the empirical mean is calculated as

$$\bar{x} = \frac{1}{k+1}x^{(k)} + \frac{k}{k+1}\bar{x}^{k-1}$$

This is done after a certain threshold value, k_0.

The 1D and 2D marginal plots for AM is shown below in Figure 6.

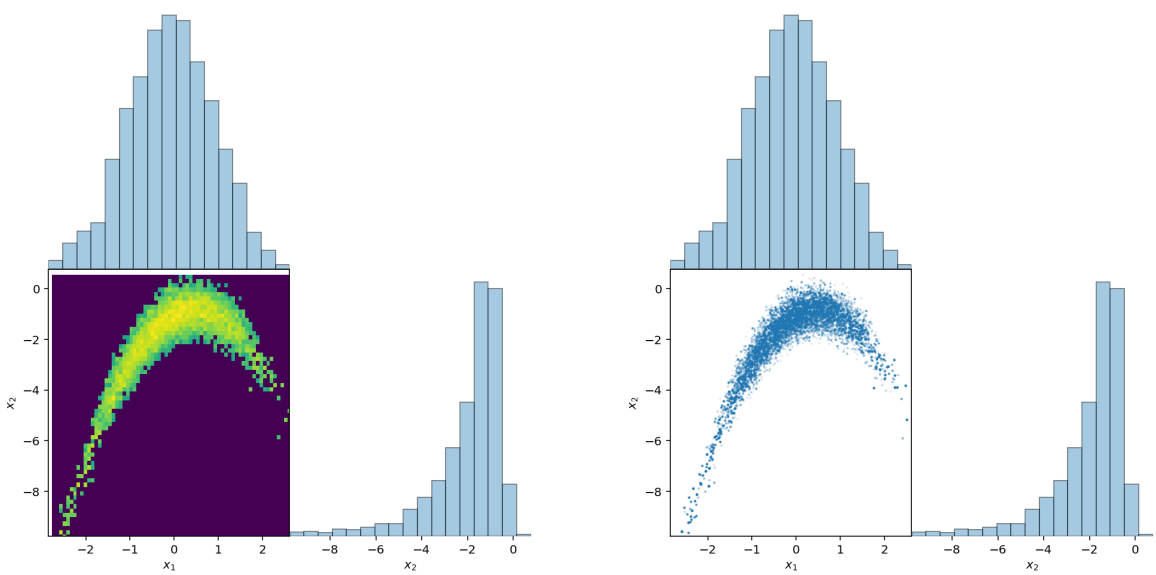

Figure 6: 1D and 2D Marginal Plots for AM

The autocorrelation plot is shown below in Figure 7.

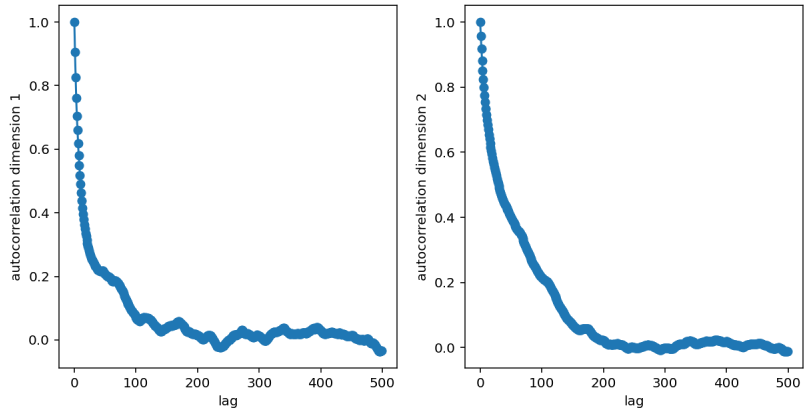

Figure 7: Autocorrelation Plots for AM

The integrated autocorrelation value is 69.02 for the first dimension and 114.36 for the second dimension.

The acceptance ratio is 0.21.

Visual inspection of mixing is shown below in Figure 8.

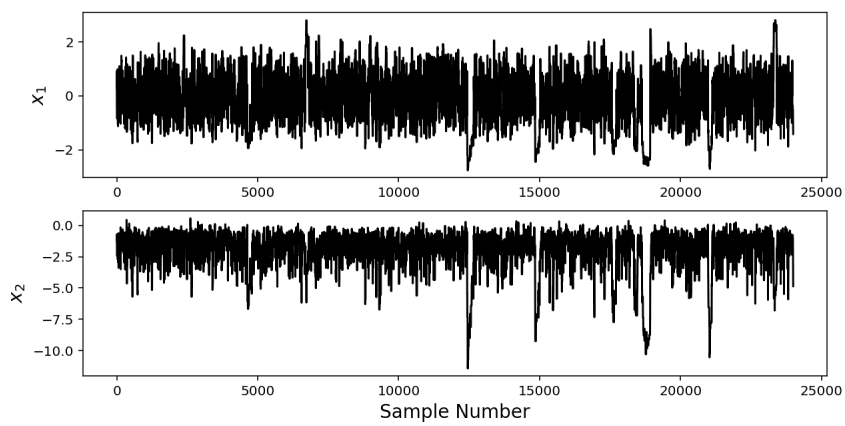

Figure 8: Visual Inspection of Mixing for AM

1.3. Delayed Rejection Metropolis Hastings MCMC (DR)

DR algorithm, in the scope of this project, basically proposes a second stage move instead of staying in the same position in the case of rejection of the first proposal. In addition to the first candidate move y_1 in MH, another proposal $y_2 \sim (q_2(y_2|x, y_1))$ is made with the acceptance probability

$$a_2(x, y_1, y_2) = min\left(1, \frac{f_X(y_2)q_1(y_1|y_2)q_2(x|y_2, y_1)(1 - a_1(y_2, y_1))}{f_X(x)q_1(y_1|x)q_2(y_2|y_1, x)(1 - a_1(x, y_1))}\right)$$

Here, q_2's cancel whereas q_1's don't.

The 1D and 2D marginal plots for DR is shown below in Figure 9.

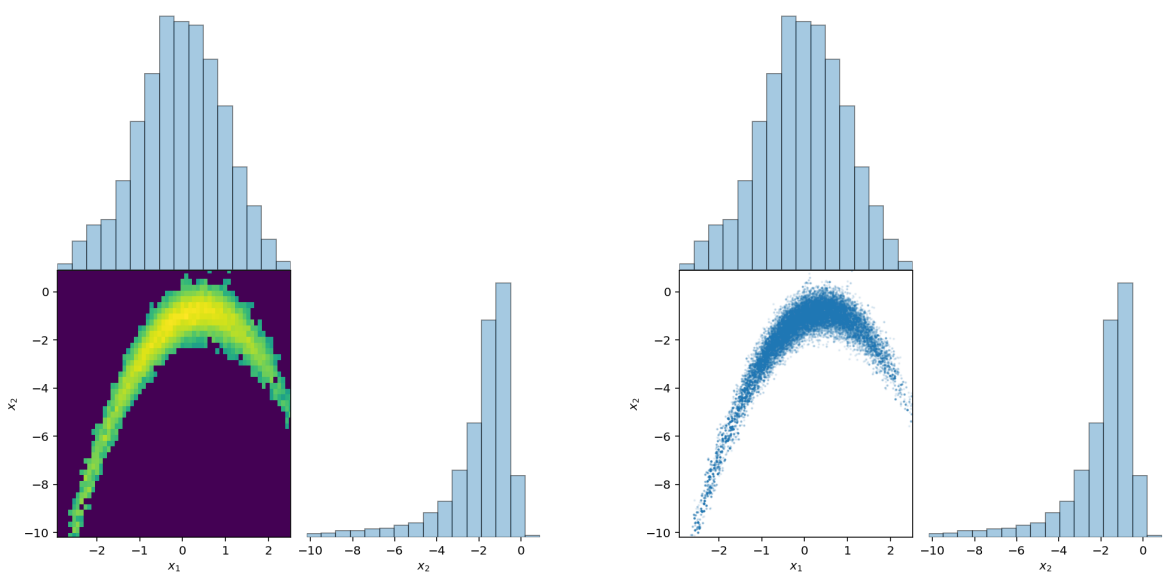

Figure 9: 1D and 2D Marginal Plots for DR

The autocorrelation plot is shown below in Figure 10.

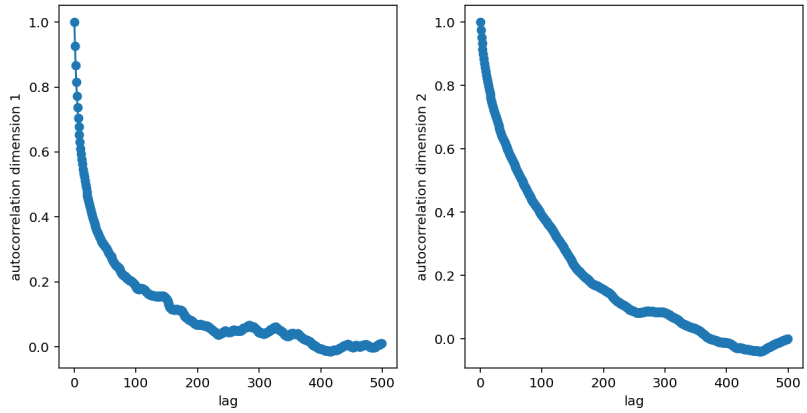

Figure 10: Autocorrelation Plots for DR

The integrated autocorrelation value is 117.95 for the first dimension and 194.39 for the second dimension.

The acceptance ratio is 0.27.

Visual inspection of mixing is shown below in Figure 11.

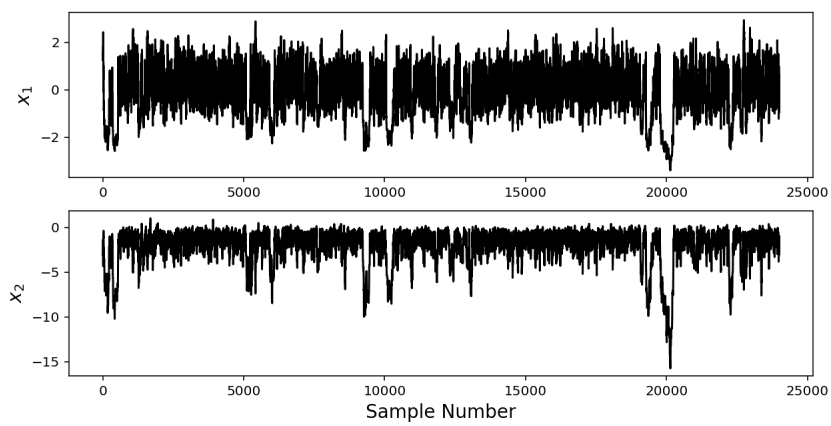

Figure 11: Visual Inspection of Mixing for DR

1.4. Delayed Rejection Adaptive Metropolis Hastings MCMC (DRAM)

In this method, AM and DR methods which are explained above are combined. The 1D and 2D marginal plots for DRAM is shown below in Figure 12.

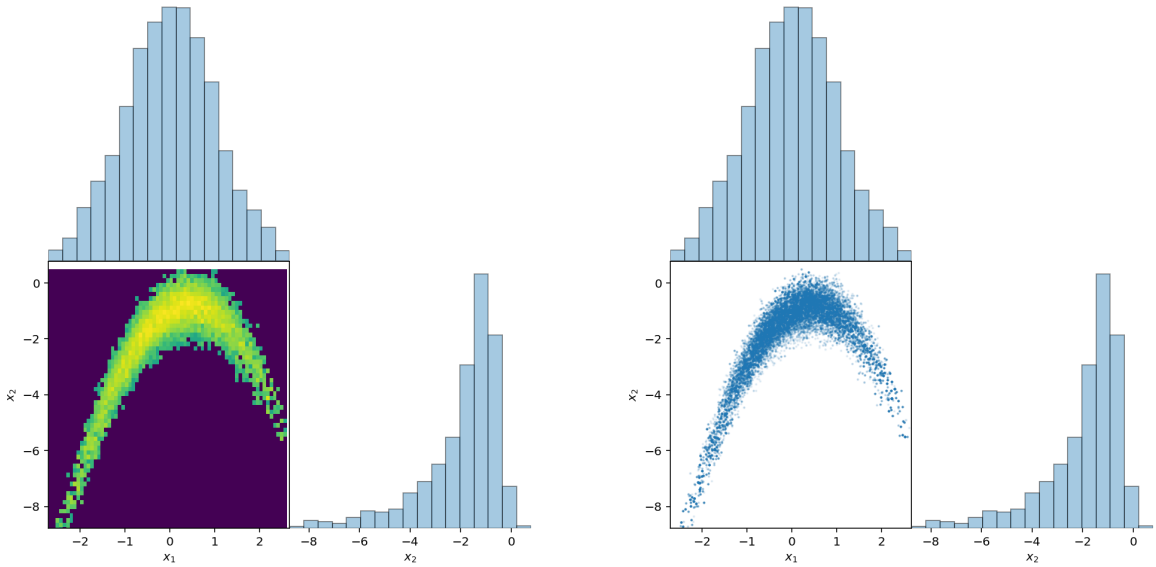

Figure 12: 1D and 2D Marginal Plots for DRAM

The autocorrelation plot is shown below in Figure 13.

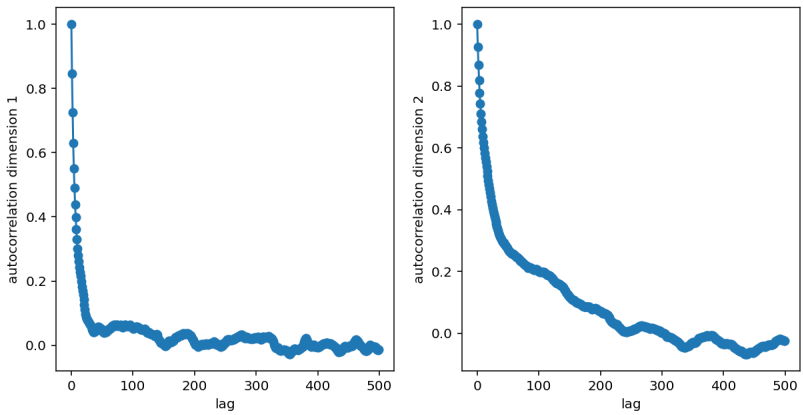

Figure 13: Autocorrelation Plots for DRAM

The integrated autocorrelation value is 34.42 for the first dimension and 88.58 for the second dimension.

The acceptance ratio is 0.32.

Visual inspection of mixing is shown below in Figure 14.

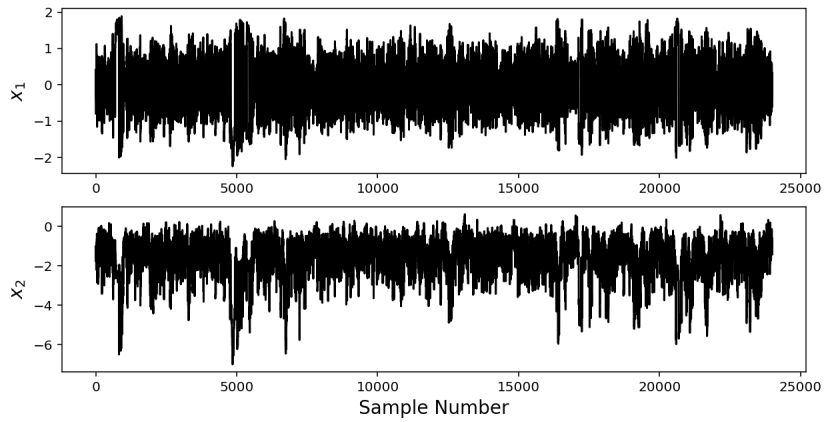

Figure 14: Visual Inspection of Mixing for DRAM

1.5. Comparison

The comparison for these algorithms can be done by examining the visual outputs as well as the acceptance ratios. The acceptance ratios are not immensely different, however DRAM has the highest acceptance ratio.

Table 1: Acceptance Ratios

	MH	AM	DR	DRAM
Acceptance Ratio	0.26	0.21	0.27	0.32

Autocorrelation graphs show the correlation between each sample set evolving in time. So, the faster/rapid this autocorrelation value goes to zero, the better job a sampling algorithm is doing in order to sample with less dependance/ relation on the previous samples. So, faster decay means that the algorithm is better.

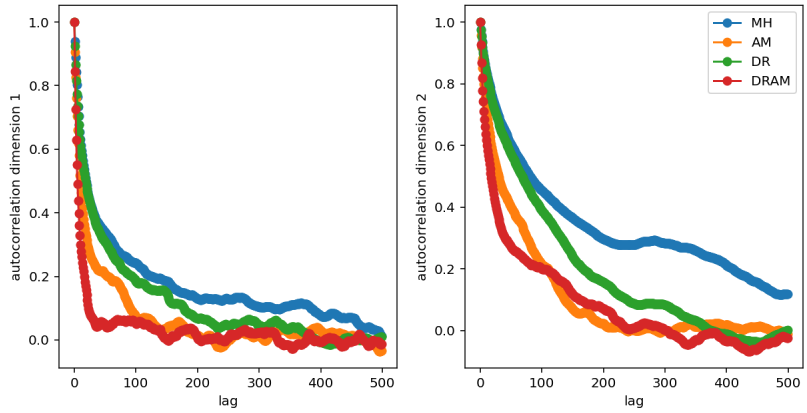

Figure 15: Autocorrelation Plots for the Algorithms

It can be seen from Figure 15 that DRAM has the fastest decay, whereas MH has the slowest decay. This would suggest that they are the best and the worst algorithms respectively.

Integrated autocorrelation (IAC) can also be used to comment on the efficiency of the algorithm. The higher the value is, the worse the algorithm is since it is basically the autocorrelation values summed up. The IAC values shown on Table 2 also supports that DRAM is the best algorithm whereas MH is the worst. DRAM lowered IAC values drastically, and MH had distinctively high IAC values.

Table 2: IAC Values

	MH		AM		DR		DRAM	
	Dim 1	Dim 2	Dim 1	Dim 2	Dim 1	Dim 2	Dim 1	Dim 2
IAC	254.85	361.88	69.02	114.36	117.95	194.39	34.42	88.58

The visual inspections of these algorithms show the samples acquired with each algorithm. So, it can be interpreted that sampling is done better when the samples are more random, varying in a random manner rather than having spikes and trends, like in the white noise. As it can be seen in Figure 14, DRAM has its samples looking the most like white noise, so this also supports that DRAM is a better algorithm.

2. Truth Model

SIR (Susceptible, Infected and Recovered) model of a disease in a population is governed by the differential equations given below for each section. The total population is 1000 and the initial value for each state is as shown below for both of the cases.

$$N = 1000$$

$$S(0) = 900$$

$$I(0) = 100$$

$$R(0) = 0$$

This problem considers the evolution of SIR portions of the population in the timescale:

$$t \in [0,6]$$

$$t = linspace(0,6,61)$$

The differential equations are solved with the Runge-Kutta 45 method for each section to compute the evolution. The state of interest here is I (Infected). Data from that is created by adding noise to it.

$$D = I + \xi \qquad , where \quad \xi \sim N(0,1)$$

2.1. Identifiable Version

SIR (Susceptible, Infected and Recovered) model of a disease in a population in the identifiable case is governed by the differential equations given below.

$$\frac{dS}{dt} = \delta N - \delta S - \beta IS$$

$$\frac{dI}{dt} = \beta IS = (r + \delta)I$$

$$\frac{dR}{dt} = rI - \delta R$$

In the identifiable version of this model, the true values of the parameters

$$\theta = (\beta, r, \delta) = (0.02, 0.6, 0.15)$$

The states are evolving as shown in Figure 15 In this case.

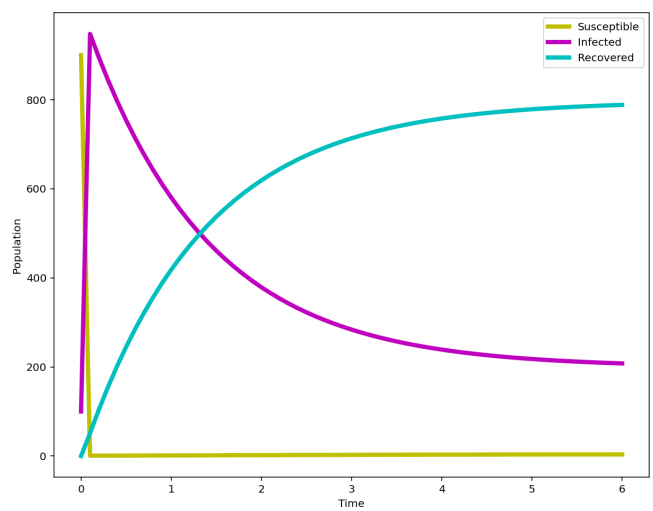

Figure 15

The data created from the I state, I_{data}, computed is as shown below.

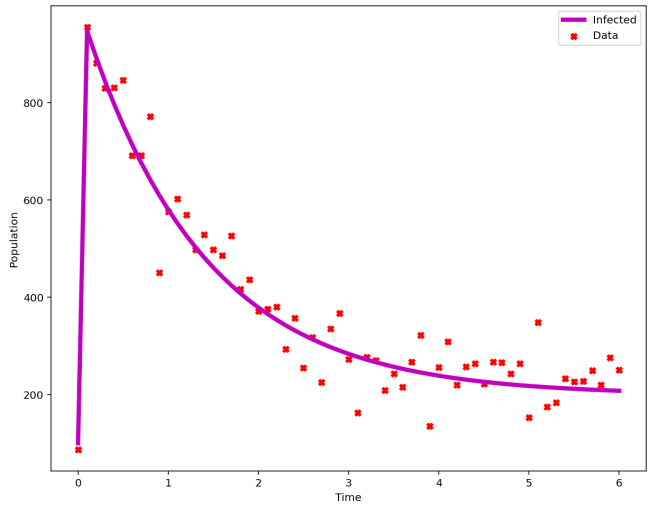

Figure 16

2.2. Non-Identifiable Version

SIR (Susceptible, Infected and Recovered) model of a disease in a population in the identifiable case is governed by the differential equations given below.

$$\frac{dS}{dt} = \delta N - \delta S - \gamma \kappa IS$$

$$\frac{dI}{dt} = \gamma \kappa IS = (r + \delta)I$$

$$\frac{dR}{dt} = rI - \delta R$$

In the non-identifiable version of this model, the true values of the parameters

$$\theta = (\gamma, \kappa, r, \delta) = (0.2,0.1,0.6,0.15)$$

The states are evolving as shown in Figure 17 In this case.

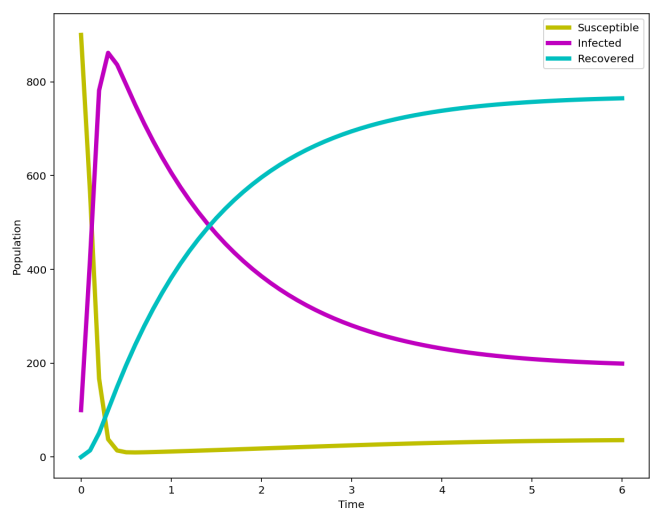

Figure 17

The data created from the I state, I_{data}, computed is as shown below.

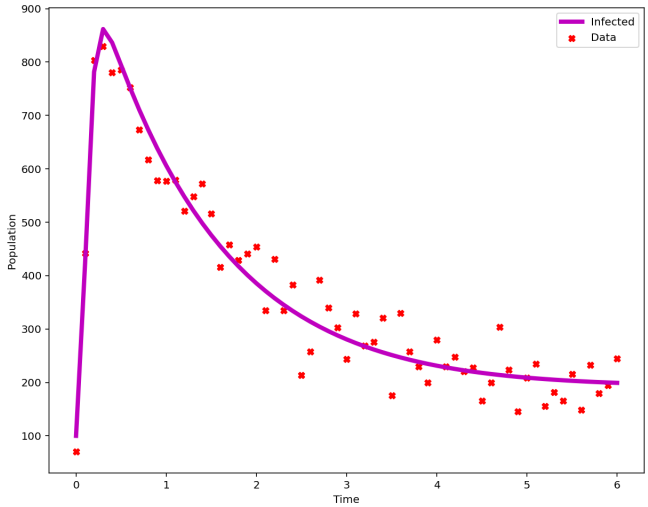
Infected
Data
Population
Time
900
800
700
600
500
400
300
200
100
0
1
2
3
4
5
6

Figure 18

3. DRAM for Bayesian Inference

In this section, the works done above are combined. Data created, $I_{data} = \{d_1, d_2, \ldots, d_i\}$, is used with the DRAM algorithm to learn the SIR model parameters. The prior for the parameters are Gaussian as

$$\theta \sim N(0,1)$$

$$f_\theta(\theta) = \prod_{i=1}^{k} f_\theta(\theta_i),\ where \quad k = 3\ for\ Section\ 3.1\ and\ k = 4\ for\ Section\ 3.2$$

The likelihood function is constructed as a log likelihood. The log of the prior is taken and summed with the likelihood to obtain posterior.

$$d_i \sim N(y_i, std),\ where \quad std = 50$$

$$f_D(D\,|\,\theta) = \prod_{i=1}^{k=61} f_D(d_i\,|\,\theta)$$

$$f_{\theta|D}(\theta\,|\,d) \sim f_D(D\,|\,\theta) f_\theta(\theta)$$

3.1. Identifiable Version

In this section, data created in section 2.1 is used with DRAM algorithm. The algorithm has an acceptance ratio of 0.32.

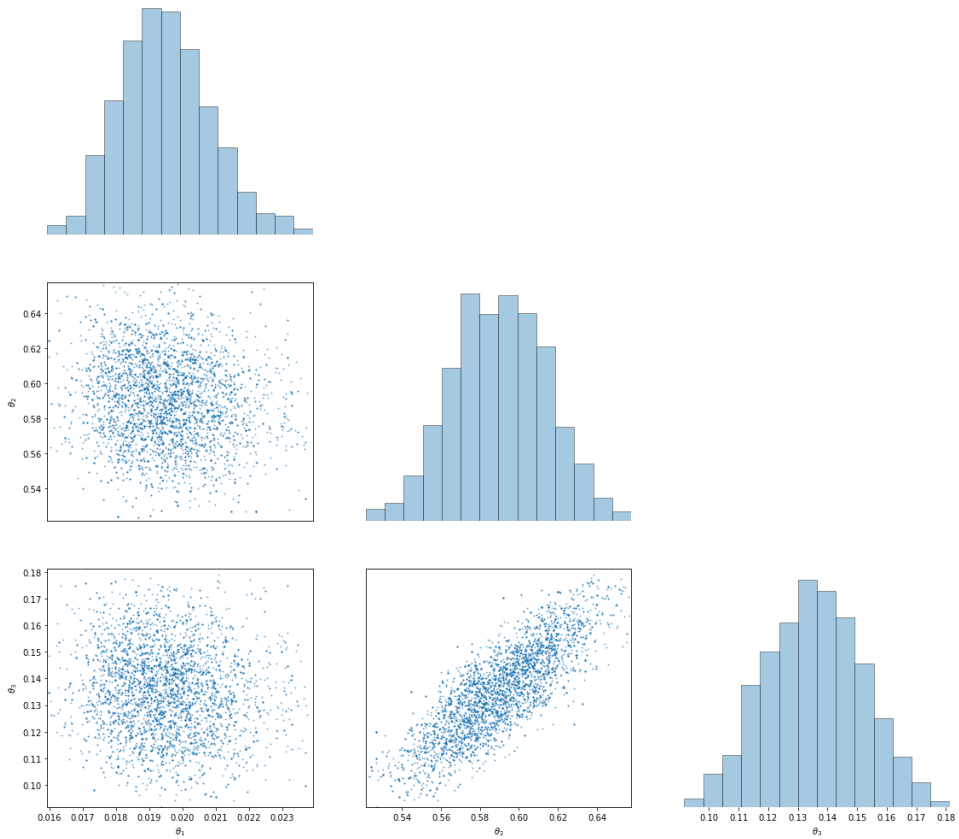

Figure 19: 1D and 2D Marginal Plots for the Identifiable Version

The IAC values are 10.03, 29.63, and 33.62 for the first (β), second (r), and third (δ) dimension respectively. The autocorrelation plots for the can be seen in Figure 20.

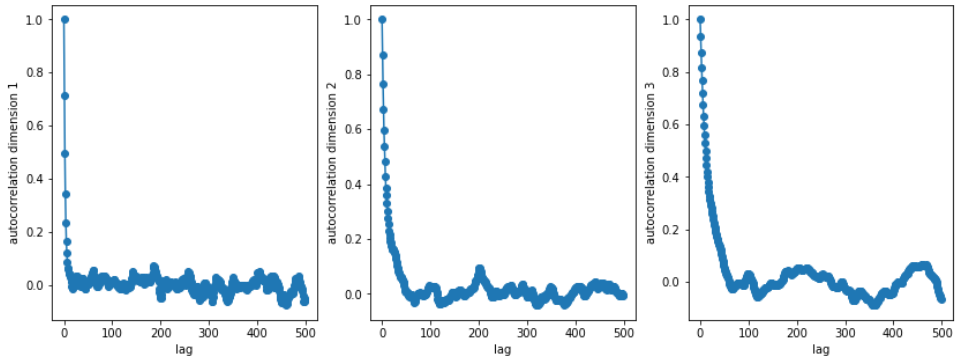

Figure 20: Autocorrelation Plots for the Identifiable Version

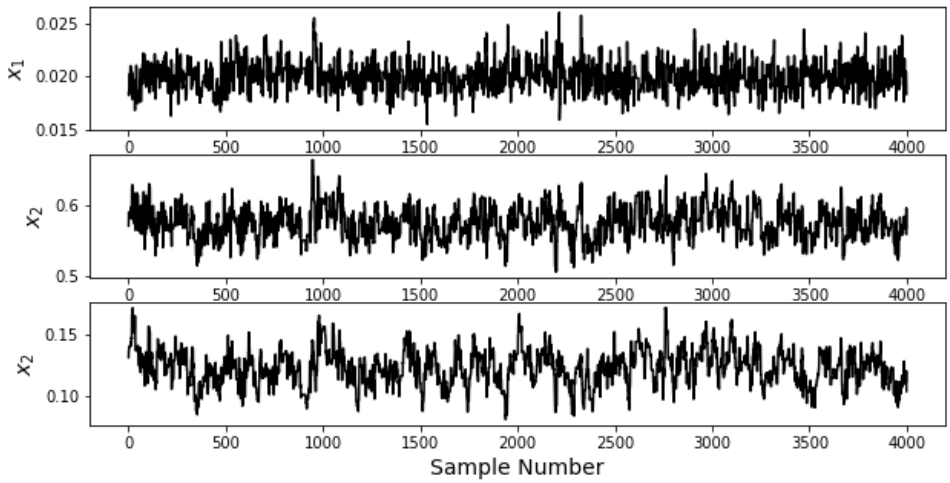

Figure 21: Visual Inspection of Mixing for the Identifiable Version

3.2.Non-Identifiable Version

In this section, data created in section 2.2 is used with DRAM algorithm. The algorithm has an acceptance ratio of 0.04.

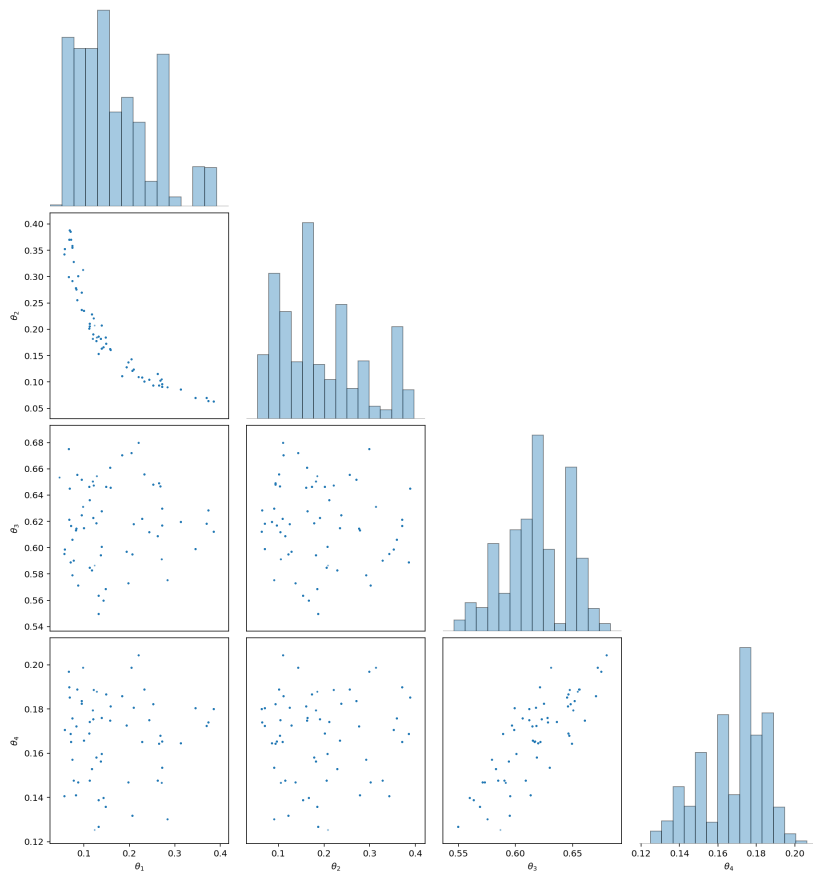

Figure 22: 1D and 2D Marginal Plots for the Non-Identifiable Version

The IAC values are 131.42, 62.97, 491.13 and 221.29 for the first (γ), second(κ), third(r), and fourth (δ) dimension respectively. The autocorrelation plots for the can be seen in Figure 23.

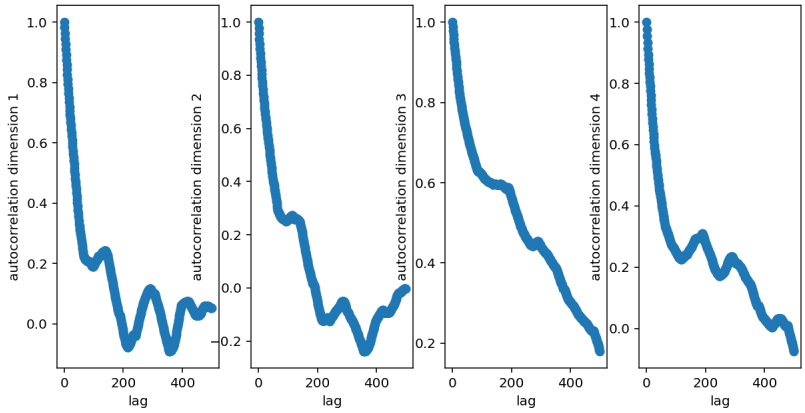

Figure 23: Autocorrelation Plots for the Non-Identifiable Version

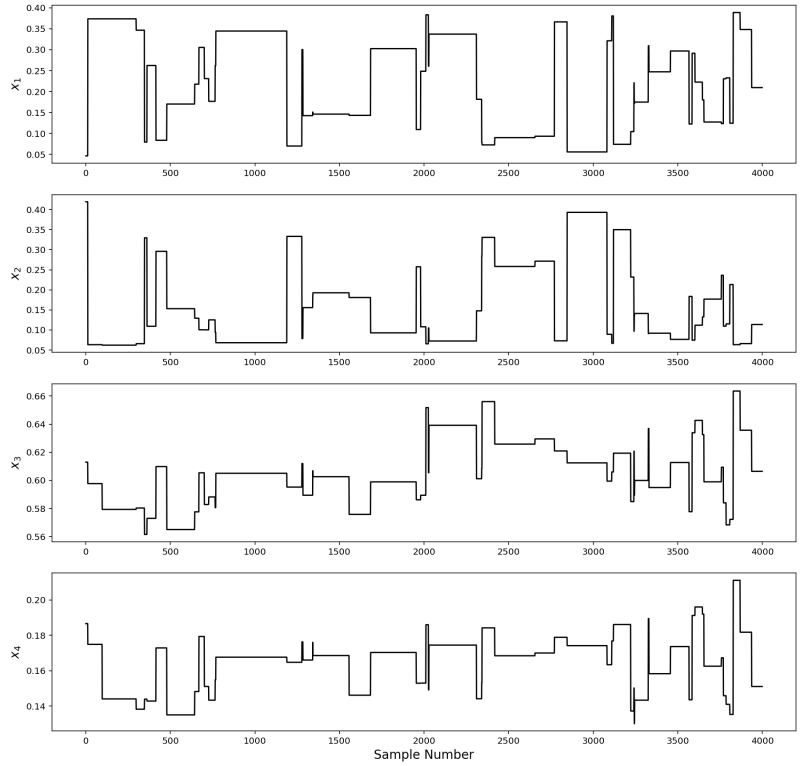

Figure 24: Visual Inspection of Mixing for the Non-Identifiable Version

Non-Identifiable case does not seem to be properly working, so it is hard to comment on it. But from the identifiable case, it could be seen from Figure 19 that the parameters are close to the original values. Also, the autocorrelation graphs in Figure 20 is decreasing rapidly, whereas it looks much more slower in Figure 23. The samples also look like white noise in the identifiable case as shown in Figure 21. Since the parameters are non-identifiable in the second case, these can be changed and the effect of this on the differential equation may not be observable if their multiplication is as desired. Hence, it is harder to infer their values. It can be seen from Figure 22 top left graph between θ_1, (γ), and θ_2, (κ), it is basically like a curve where $\gamma = \frac{\beta}{\kappa}$, since $\gamma\kappa$ is used in the differential equations instead of β, and this curve parameters multiplied fits into to the range its values.

4. Appendices

```
<sampling_size>
<sub_sample_data_samples>
<sub_sample_data_labels>
<sub_sample_data_features_1>
<sub_sample_data_features_2>
<sub_sample_data_features_3>
<sub_sample_data_features_4>
<sub_sample_data_features_5>
<sub_sample_data_features_6>
<sub_sample_data_features_7>
<sub_sample_data_features_8>
<sub_sample_data_features_9>
<sub_sample_data_features_10>
<sub_sample_data_features_11>
<sub_sample_data_features_12>
<sub_sample_data_features_13>
<sub_sample_data_features_14>
<sub_sample_data_features_15>
<sub_sample_data_features_16>
<sub_sample_data_features_17>
<sub_sample_data_features_18>
<sub_sample_data_features_19>
<sub_sample_data_features_20>
<sub_sample_data_features_21>
<sub_sample_data_features_22>
<sub_sample_data_features_23>
<sub_sample_data_features_24>
<sub_sample_data_features_25>
<sub_sample_data_features_26>
<sub_sample_data_features_27>
<sub_sample_data_features_28>
<sub_sample_data_features_29>
<sub_sample_data_features_30>
<sub_sample_data_features_31>
<sub_sample_data_features_32>
<sub_sample_data_features_33>
<sub_sample_data_features_34>
<sub_sample_data_features_35>
<sub_sample_data_features_36>
<sub_sample_data_features_37>
<sub_sample_data_features_38>
<sub_sample_data_features_39>
<sub_sample_data_features_40>
<sub_sample_data_features_41>
<sub_sample_data_features_42>
<sub_sample_data_features_43>
<sub_sample_data_features_44>
<sub_sample_data_features_45>
<sub_sample_data_features_46>
<sub_sample_data_features_47>
<sub_sample_data_features_48>
<sub_sample_data_features_49>
<sub_sample_data_features_50>
<sub_sample_data_features_51>
<sub_sample_data_features_52>
<sub_sample_data_features_53>
<sub_sample_data_features_54>
<sub_sample_data_features_55>
<sub_sample_data_features_56>
<sub_sample_data_features_57>
<sub_sample_data_features_58>
<sub_sample_data_features_59>
<sub_sample_data_features_60>
<sub_sample_data_features_61>
<sub_sample_data_features_62>
<sub_sample_data_features_63>
<sub_sample_data_features_64>
<sub_sample_data_features_65>
<sub_sample_data_features_66>
<sub_sample_data_features_67>
<sub_sample_data_features_68>
<sub_sample_data_features_69>
<sub_sample_data_features_70>
<sub_sample_data_features_71>
<sub_sample_data_features_72>
<sub_sample_data_features_73>
<sub_sample_data_features_74>
<sub_sample_data_features_75>
<sub_sample_data_features_76>
<sub_sample_data_features_77>
<sub_sample_data_features_78>
<sub_sample_data_features_79>
<sub_sample_data_features_80>
<sub_sample_data_features_81>
<sub_sample_data_features_82>
<sub_sample_data_features_83>
<sub_sample_data_features_84>
<sub_sample_data_features_85>
<sub_sample_data_features_86>
<sub_sample_data_features_87>
<sub_sample_data_features_88>
<sub_sample_data_features_89>
<sub_sample_data_features_90>
<sub_sample_data_features_91>
<sub_sample_data_features_92>
<sub_sample_data_features_93>
<sub_sample_data_features_94>
<sub_sample_data_features_95>
<sub_sample_data_features_96>
<sub_sample_data_features_97>
<sub_sample_data_features_98>
<sub_sample_data_features_99>
<sub_sample_data_features_100>
<sub_sample_data_features_101>
<sub_sample_data_features_102>
<sub_sample_data_features_103>
<sub_sample_data_features_104>
<sub_sample_data_features_105>
<sub_sample_data_features_106>
<sub_sample_data_features_107>
<sub_sample_data_features_108>
<sub_sample_data_features_109>
<sub_sample_data_features_110>
<sub_sample_data_features_111>
<sub_sample_data_features_112>
<sub_sample_data_features_113>
<sub_sample_data_features_114>
<sub_sample_data_features_115>
<sub_sample_data_features_116>
<sub_sample_data_features_117>
<sub_sample_data_features_118>
<sub_sample_data_features_119>
<sub_sample_data_features_120>
<sub_sample_data_features_121>
<sub_sample_data_features_122>
<sub_sample_data_features_123>
<sub_sample_data_features_124>
<sub_sample_data_features_125>
<sub_sample_data_features_126>
<sub_sample_data_features_127>
<sub_sample_data_features_128>
<sub_sample_data_features_129>
<sub_sample_data_features_130>
<sub_sample_data_features_131>
<sub_sample_data_features_132>
<sub_sample_data_features_133>
<sub_sample_data_features_134>
<sub_sample_data_features_135>
<sub_sample_data_features_136>
<sub_sample_data_features_137>
<sub_sample_data_features_138>
<sub_sample_data_features_139>
<sub_sample_data_features_140>
<sub_sample_data_features_141>
<sub_sample_data_features_142>
<sub_sample_data_features_143>
<sub_sample_data_features_144>
<sub_sample_data_features_145>
<sub_sample_data_features_146>
<sub_sample_data_features_147>
<sub_sample_data_features_148>
<sub_sample_data_features_149>
<sub_sample_data_features_150>
<sub_sample_data_features_151>
<sub_sample_data_features_152>
<sub_sample_data_features_153>
<sub_sample_data_features_154>
<sub_sample_data_features_155>
<sub_sample_data_features_156>
<sub_sample_data_features_157>
<sub_sample_data_features_158>
<sub_sample_data_features_159>
<sub_sample_data_features_160>
<sub_sample_data_features_161>
<sub_sample_data_features_162>
<sub_sample_data_features_163>
<sub_sample_data_features_164>
<sub_sample_data_features_165>
<sub_sample_data_features_166>
<sub_sample_data_features_167>
<sub_sample_data_features_168>
<sub_sample_data_features_169>
<sub_sample_data_features_170>
<sub_sample_data_features_171>
<sub_sample_data_features_172>
<sub_sample_data_features_173>
<sub_sample_data_features_174>
<sub_sample_data_features_175>
<sub_sample_data_features_176>
<sub_sample_data_features_177>
<sub_sample_data_features_178>
<sub_sample_data_features_179>
<sub_sample_data_features_180>
<sub_sample_data_features_181>
<sub_sample_data_features_182>
<sub_sample_data_features_183>
<sub_sample_data_features_184>
<sub_sample_data_features_185>
<sub_sample_data_features_186>
<sub_sample_data_features_187>
<sub_sample_data_features_188>
<sub_sample_data_features_189>
<sub_sample_data_features_190>
<sub_sample_data_features_191>
<sub_sample_data_features_192>
<sub_sample_data_features_193>
<sub_sample_data_features_194>
<sub_sample_data_features_195>
<sub_sample_data_features_196>
<sub_sample_data_features_197>
<sub_sample_data_features_198>
<sub_sample_data_features_199>
<sub_sample_data_features_200>
<sub_sample_data_features_201>
<sub_sample_data_features_202>
<sub_sample_data_features_203>
<sub_sample_data_features_204>
<sub_sample_data_features_205>
<sub_sample_data_features_206>
<sub_sample_data_features_207>
<sub_sample_data_features_208>
<sub_sample_data_features_209>
<sub_sample_data_features_210>
<sub_sample_data_features_211>
<sub_sample_data_features_212>
<sub_sample_data_features_213>
<sub_sample_data_features_214>
<sub_sample_data_features_215>
<sub_sample_data_features_216>
<sub_sample_data_features_217>
<sub_sample_data_features_218>
<sub_sample_data_features_219>
<sub_sample_data_features_220>
<sub_sample_data_features_221>
<sub_sample_data_features_222>
<sub_sample_data_features_223>
<sub_sample_data_features_224>
<sub_sample_data_features_225>
<sub_sample_data_features_226>
<sub_sample_data_features_227>
<sub_sample_data_features_228>
<sub_sample_data_features_229>
<sub_sample_data_features_230>
<sub_sample_data_features_231>
<sub_sample_data_features_232>
<sub_sample_data_features_233>
<sub_sample_data_features_234>
<sub_sample_data_features_235>
<sub_sample_data_features_236>
<sub_sample_data_features_237>
<sub_sample_data_features_238>
<sub_sample_data_features_239>
<sub_sample_data_features_240>
<sub_sample_data_features_241>
<sub_sample_data_features_242>
<sub_sample_data_features_243>
<sub_sample_data_features_244>
<sub_sample_data_features_245>
<sub_sample_data_features_246>
<sub_sample_data_features_247>
<sub_sample_data_features_248>
<sub_sample_data_features_249>
<sub_sample_data_features_250>
<sub_sample_data_features_251>
<sub_sample_data_features_252>
<sub_sample_data_features_253>
<sub_sample_data_features_254>
<sub_sample_data_features_255>
<sub_sample_data_features_256>
<sub_sample_data_features_257>
<sub_sample_data_features_258>
<sub_sample_data_features_259>
<sub_sample_data_features_260>
<sub_sample_data_features_261>
<sub_sample_data_features_262>
<sub_sample_data_features_263>
<sub_sample_data_features_264>
<sub_sample_data_features_265>
<sub_sample_data_features_266>
<sub_sample_data_features_267>
<sub_sample_data_features_268>
<sub_sample_data_features_269>
<sub_sample_data_features_270>
<sub_sample_data_features_271>
<sub_sample_data_features_272>
<sub_sample_data_features_273>
<sub_sample_data_features_274>
<sub_sample_data_features_275>
<sub_sample_data_features_276>
<sub_sample_data_features_277>
<sub_sample_data_features_278>
<sub_sample_data_features_279>
<sub_sample_data_features_280>
<sub_sample_data_features_281>
<sub_sample_data_features_282>
<sub_sample_data_features_283>
<sub_sample_data_features_284>
<sub_sample_data_features_285>
<sub_sample_data_features_286>
<sub_sample_data_features_287>
<sub_sample_data_features_288>
<sub_sample_data_features_289>
<sub_sample_data_features_290>
<sub_sample_data_features_291>
<sub_sample_data_features_292>
<sub_sample_data_features_293>
<sub_sample_data_features_294>
<sub_sample_data_features_295>
<sub_sample_data_features_296>
<sub_sample_data_features_297>
<sub_sample_data_features_298>
<sub_sample_data_features_299>
<sub_sample_data_features_300>
<sub_sample_data_features_301>
<sub_sample_data_features_302>
<sub_sample_data_features_303>
<sub_sample_data_features_304>
<sub_sample_data_features_305>
<sub_sample_data_features_306>
<sub_sample_data_features_307>
<sub_sample_data_features_308>
<sub_sample_data_features_309>
<sub_sample_data_features_310>
<sub_sample_data_features_311>
<sub_sample_data_features_312>
<sub_sample_data_features_313>
<sub_sample_data_features_314>
<sub_sample_data_features_315>
<sub_sample_data_features_316>
<sub_sample_data_features_317>
<sub_sample_data_features_318>
<sub_sample_data_features_319>
<sub_sample_data_features_320>
<sub_sample_data_features_321>
<sub_sample_data_features_322>
<sub_sample_data_features_323>
<sub_sample_data_features_324>
<sub_sample_data_features_325>
<sub_sample_data_features_326>
<sub_sample_data_features_327>
<sub_sample_data_features_328>
<sub_sample_data_features_329>
<sub_sample_data_features_330>
<sub_sample_data_features_331>
<sub_sample_data_features_332>
<sub_sample_data_features_333>
<sub_sample_data_features_334>
<sub_sample_data_features_335>
<sub_sample_data_features_336>
<sub_sample_data_features_337>
<sub_sample_data_features_338>
<sub_sample_data_features_339>
<sub_sample_data_features_340>
<sub_sample_data_features_341>
<sub_sample_data_features_342>
<sub_sample_data_features_343>
<sub_sample_data_features_344>
<sub_sample_data_features_345>
<sub_sample_data_features_346>
<sub_sample_data_features_347>
<sub_sample_data_features_348>
<sub_sample_data_features_349>
<sub_sample_data_features_350>
<sub_sample_data_features_351>
<sub_sample_data_features_352>
<sub_sample_data_features_353>
<sub_sample_data_features_354>
<sub_sample_data_features_355>
<sub_sample_data_features_356>
<sub_sample_data_features_357>
<sub_sample_data_features_358>
<sub_sample_data_features_359>
<sub_sample_data_features_360>
<sub_sample_data_features_361>
<sub_sample_data_features_362>
<sub_sample_data_features_363>
<sub_sample_data_features_364>
<sub_sample_data_features_365>
<sub_sample_data_features_366>
<sub_sample_data_features_367>
<sub_sample_data_features_368>
<sub_sample_data_features_369>
<sub_sample_data_features_370>
<sub_sample_data_features_371>
<sub_sample_data_features_372>
<sub_sample_data_features_373>
<sub_sample_data_features_374>
<sub_sample_data_features_375>
<sub_sample_data_features_376>
<sub_sample_data_features_377>
<sub_sample_data_features_378>
<sub_sample_data_features_379>
<sub_sample_data_features_380>
<sub_sample_data_features_381>
<sub_sample_data_features_382>
<sub_sample_data_features_383>
<sub_sample_data_features_384>
<sub_sample_data_features_385>
<sub_sample_data_features_386>
<sub_sample_data_features_387>
<sub_sample_data_features_388>
<sub_sample_data_features_389>
<sub_sample_data_features_390>
<sub_sample_data_features_391>
<sub_sample_data_features_392>
<sub_sample_data_features_393>
<sub_sample_data_features_394>
<sub_sample_data_features_395>
<sub_sample_data_features_396>
<sub_sample_data_features_397>
<sub_sample_data_features_398>
<sub_sample_data_features_399>
<sub_sample_data_features_400>
<sub_sample_data_features_401>
<sub_sample_data_features_402>
<sub_sample_data_features_403>
<sub_sample_data_features_404>
<sub_sample_data_features_405>
<sub_sample_data_features_406>
<sub_sample_data_features_407>
<sub_sample_data_features_408>
<sub_sample_data_features_409>
<sub_sample_data_features_410>
<sub_sample_data_features_411>
<sub_sample_data_features_412>
<sub_sample_data_features_413>
<sub_sample_data_features_414>
<sub_sample_data_features_415>
<sub_sample_data_features_416>
<sub_sample_data_features_417>
<sub_sample_data_features_418>
<sub_sample_data_features_419>
<sub_sample_data_features_420>
<sub_sample_data_features_421>
<sub_sample_data_features_422>
<sub_sample_data_features_423>
<sub_sample_data_features_424>
<sub_sample_data_features_425>
<sub_sample_data_features_426>
<sub_sample_data_features_427>
<sub_sample_data_features_428>
<sub_sample_data_features_429>
<sub_sample_data_features_430>
<sub_sample_data_features_431>
<sub_sample_data_features_432>
<sub_sample_data_features_433>
<sub_sample_data_features_434>
<sub_sample_data_features_435>
<sub_sample_data_features_436>
<sub_sample_data_features_437>
<sub_sample_data_features_438>
<sub_sample_data_features_439>
<sub_sample_data_features_440>
<sub_sample_data_features_441>
<sub_sample_data_features_442>
<sub_sample_data_features_443>
<sub_sample_data_features_444>
<sub_sample_data_features_445>
<sub_sample_data_features_446>
<sub_sample_data_features_447>
<sub_sample_data_features_448>
<sub_sample_data_features_449>
<sub_sample_data_features_450>
<sub_sample_data_features_451>
<sub_sample_data_features_452>
<sub_sample_data_features_453>
<sub_sample_data_features_454>
<sub_sample_data_features_455>
<sub_sample_data_features_456>
<sub_sample_data_features_457>
<sub_sample_data_features_458>
<sub_sample_data_features_459>
<sub_sample_data_features_460>
<sub_sample_data_features_461>
<sub_sample_data_features_462>
<sub_sample_data_features_463>
<sub_sample_data_features_464>
<sub_sample_data_features_465>
<sub_sample_data_features_466>
<sub_sample_data_features_467>
<sub_sample_data_features_468>
<sub_sample_data_features_469>
<sub_sample_data_features_470>
<sub_sample_data_features_471>
<sub_sample_data_features_472>
<sub_sample_data_features_473>
<sub_sample_data_features_474>
<sub_sample_data_features_475>
<sub_sample_data_features_476>
<sub_sample_data_features_477>
<sub_sample_data_features_478>
<sub_sample_data_features_479>
<sub_sample_data_features_480>
<sub_sample_data_features_481>
<sub_sample_data_features_482>
<sub_sample_data_features_483>
<sub_sample_data_features_484>
<sub_sample_data_features_485>
<sub_sample_data_features_486>
<sub_sample_data_features_487>
<sub_sample_data_features_488>
<sub_sample_data_features_489>
<sub_sample_data_features_490>
<sub_sample_data_features_491>
<sub_sample_data_features_492>
<sub_sample_data_features_493>
<sub_sample_data_features_494>
<sub_sample_data_features_495>
<sub_sample_data_features_496>
<sub_sample_data_features_497>
<sub_sample_data_features_498>
<sub_sample_data_features_499>
<sub_sample_data_features_500>
<sub_sample_data_features_501>
<sub_sample_data_features_502>
<sub_sample_data_features_503>
<sub_sample_data_features_504>
<sub_sample_data_features_505>
<sub_sample_data_features_506>
<sub_sample_data_features_507>
<sub_sample_data_features_508>
<sub_sample_data_features_509>
<sub_sample_data_features_510>
<sub_sample_data_features_511>
<sub_sample_data_features_512>
<sub_sample_data_features_513>
<sub_sample_data_features_514>
<sub_sample_data_features_515>
<sub_sample_data_features_516>
<sub_sample_data_features_517>
<sub_sample_data_features_518>
<sub_sample_data_features_519>
<sub_sample_data_features_520>
<sub_sample_data_features_521>
<sub_sample_data_features_522>
<sub_sample_data_features_523>
<sub_sample_data_features_524>
<sub_sample_data_features_525>
<sub_sample_data_features_526>
<sub_sample_data_features_527>
<sub_sample_data_features_528>
<sub_sample_data_features_529>
<sub_sample_data_features_530>
<sub_sample_data_features_531>
<sub_sample_data_features_532>
<sub_sample_data_features_533>
<sub_sample_data_features_534>
<sub_sample_data_features_535>
<sub_sample_data_features_536>
<sub_sample_data_features_537>
<sub_sample_data_features_538>
<sub_sample_data_features_539>
<sub_sample_data_features_540>
<sub_sample_data_features_541>
<sub_sample_data_features_542>
<sub_sample_data_features_543>
<sub_sample_data_features_544>
<sub_sample_data_features_545>
<sub_sample_data_features_546>
<sub_sample_data_features_547>
<sub_sample_data_features_548>
<sub_sample_data_features_549>
<sub_sample_data_features_550>
<sub_sample_data_features_551>
<sub_sample_data_features_552>
<sub_sample_data_features_553>
<sub_sample_data_features_554>
<sub_sample_data_features_555>
<sub_sample_data_features_556>
<sub_sample_data_features_557>
<sub_sample_data_features_558>
<sub_sample_data_features_559>
<sub_sample_data_features_560>
<sub_sample_data_features_561>
<sub_sample_data_features_562>
<sub_sample_data_features_563>
<sub_sample_data_features_564>
<sub_sample_data_features_565>
<sub_sample_data_features_566>
<sub_sample_data_features_567>
<sub_sample_data_features_568>
<sub_sample_data_features_569>
<sub_sample_data_features_570>
<sub_sample_data_features_571>
<sub_sample_data_features_572>
<sub_sample_data_features_573>
<sub_sample_data_features_574>
<sub_sample_data_features_575>
<sub_sample_data_features_576>
<sub_sample_data_features_577>
<sub_sample_data_features_578>
<sub_sample_data_features_579>
<sub_sample_data_features_580>
<sub_sample_data_features_581>
<sub_sample_data_features_582>
<sub_sample_data_features_583>
<sub_sample_data_features_584>
<sub_sample_data_features_585>
<sub_sample_data_features_586>
<sub_sample_data_features_587>
<sub_sample_data_features_588>
<sub_sample_data_features_589>
<sub_sample_data_features_590>
<sub_sample_data_features_591>
<sub_sample_data_features_592>
<sub_sample_data_features_593>
<sub_sample_data_features_594>
<sub_sample_data_features_595>
<sub_sample_data_features_596>
<sub_sample_data_features_597>
<sub_sample_data_features_598>
<sub_sample_data_features_599>
<sub_sample_data_features_600>
<sub_sample_data_features_601>
<sub_sample_data_features_602>
<sub_sample_data_features_603>
<sub_sample_data_features_604>
<sub_sample_data_features_605>
<sub_sample_data_features_606>
<sub_sample_data_features_607>
<sub_sample_data_features_608>
<sub_sample_data_features_609>
<sub_sample_data_features_610>
<sub_sample_data_features_611>
<sub_sample_data_features_612>
<sub_sample_data_features_613>
<sub_sample_data_features_614>
<sub_sample_data_features_615>
<sub_sample_data_features_616>
<sub_sample_data_features_617>
<sub_sample_data_features_618>
<sub_sample_data_features_619>
<sub_sample_data_features_620>
<sub_sample_data_features_621>
<sub_sample_data_features_622>
<sub_sample_data_features_623>
<sub_sample_data_features_624>
<sub_sample_data_features_625>
<sub_sample_data_features_626>
<sub_sample_data_features_627>
<sub_sample_data_features_628>
<sub_sample_data_features_629>
<sub_sample_data_features_630>
<sub_sample_data_features_631>
<sub_sample_data_features_632>
<sub_sample_data_features_633>
<sub_sample_data_features_634>
<sub_sample_data_features_635>
<sub_sample_data_features_636>
<sub_sample_data_features_637>
<sub_sample_data_features_638>
<sub_sample_data_features_639>
<sub_sample_data_features_640>
<sub_sample_data_features_641>
<sub_sample_data_features_642>
<sub_sample_data_features_643>
<sub_sample_data_features_644>
<sub_sample_data_features_645>
<sub_sample_data_features_646>
<sub_sample_data_features_647>
<sub_sample_data_features_648>
<sub_sample_data_features_649>
<sub_sample_data_features_650>
<sub_sample_data_features_651>
<sub_sample_data_features_652>
<sub_sample_data_features_653>
<sub_sample_data_features_654>
<sub_sample_data_features_655>
<sub_sample_data_features_656>
<sub_sample_data_features_657>
<sub_sample_data_features_658>
<sub_sample_data_features_659>
<sub_sample_data_features_660>
<sub_sample_data_features_661>
<sub_sample_data_features_662>
<sub_sample_data_features_663>
<sub_sample_data_features_664>
<sub_sample_data_features_665>
<sub_sample_data_features_666>
<sub_sample_data_features_667>
<sub_sample_data_features_668>
<sub_sample_data_features_669>
<sub_sample_data_features_670>
<sub_sample_data_features_671>
<sub_sample_data_features_672>
<sub_sample_data_features_673>
<sub_sample_data_features_674>
<sub_sample_data_features_675>
<sub_sample_data_features_676>
<sub_sample_data_features_677>
<sub_sample_data_features_678>
<sub_sample_data_features_679>
<sub_sample_data_features_680>
<sub_sample_data_features_681>
<sub_sample_data_features_682>
<sub_sample_data_features_683>
<sub_sample_data_features_684>
<sub_sample_data_features_685>
<sub_sample_data_features_686>
<sub_sample_data_features_687>
<sub_sample_data_features_688>
<sub_sample_data_features_689>
<sub_sample_data_features_690>
<sub_sample_data_features_691>
<sub_sample_data_features_692>
<sub_sample_data_features_693>
<sub_sample_data_features_694>
<sub_sample_data_features_695>
<sub_sample_data_features_696>
<sub_sample_data_features_697>
<sub_sample_data_features_698>
<sub_sample_data_features_699>
<sub_sample_data_features_700>
<sub_sample_data_features_701>
<sub_sample_data_features_702>
<sub_sample_data_features_703>
<sub_sample_data_features_704>
<sub_sample_data_features_705>
<sub_sample_data_features_706>
<sub_sample_data_features_707>
<sub_sample_data_features_708>
<sub_sample_data_features_709>
<sub_sample_data_features_710>
<sub_sample_data_features_711>
<sub_sample_data_features_712>
<sub_sample_data_features_713>
<sub_sample_data_features_714>
<sub_sample_data_features_715>
<sub_sample_data_features_716>
<sub_sample_data_features_717>
<sub_sample_data_features_718>
<sub_sample_data_features_719>
<sub_sample_data_features_720>
<sub_sample_data_features_721>
<sub_sample_data_features_722>
<sub_sample_data_features_723>
<sub_sample_data_features_724>
<sub_sample_data_features_725>
<sub_sample_data_features_726>
<sub_sample_data_features_727>
<sub_sample_data_features_728>
<sub_sample_data_features_729>
<sub_sample_data_features_730>
<sub_sample_data_features_731>
<sub_sample_data_features_732>
<sub_sample_data_features_733>
<sub_sample_data_features_734>
<sub_sample_data_features_735>
<sub_sample_data_features_736>
<sub_sample_data_features_737>
<sub_sample_data_features_738>
<sub_sample_data_features_739>
<sub_sample_data_features_740>
<sub_sample_data_features_741>
<sub_sample_data_features_742>
<sub_sample_data_features_743>
<sub_sample_data_features_744>
<sub_sample_data_features_745>
<sub_sample_data_features_746>
<sub_sample_data_features_747>
<sub_sample_data_features_748>
<sub_sample_data_features_749>
<sub_sample_data_features_750>
<sub_sample_data_features_751>
<sub_sample_data_features_752>
<sub_sample_data_features_753>
<sub_sample_data_features_754>
<sub_sample_data_features_755>
<sub_sample_data_features_756>
<sub_sample_data_features_757>
<sub_sample_data_features_758>
<sub_sample_data_features_759>
<sub_sample_data_features_760>
<sub_sample_data_features_761>
<sub_sample_data_features_762>
<sub_sample_data_features_763>
<sub_sample_data_features_764>
<sub_sample_data_features_765>
<sub_sample_data_features_766>
<sub_sample_data_features_767>
<sub_sample_data_features_768>
<sub_sample_data_features_769>
<sub_sample_data_features_770>
<sub_sample_data_features_771>
<sub_sample_data_features_772>
<sub_sample_data_features_773>
<sub_sample_data_features_774>
<sub_sample_data_features_775>
<sub_sample_data_features_776>
<sub_sample_data_features_777>
<sub_sample_data_features_778>
<sub_sample_data_features_779>
<sub_sample_data_features_780>
<sub_sample_data_features_781>
<sub_sample_data_features_782>
<sub_sample_data_features_783>
<sub_sample_data_features_784>
<sub_sample_data_features_785>
<sub_sample_data_features_786>
<sub_sample_data_features_787>
<sub_sample_data_features_788>
<sub_sample_data_features_789>
<sub_sample_data_features_790>
<sub_sample_data_features_791>
<sub_sample_data_features_792>
<sub_sample_data_features_793>
<sub_sample_data_features_794>
<sub_sample_data_features_795>
<sub_sample_data_features_796>
<sub_sample_data_features_797>
<sub_sample_data_features_798>
<sub_sample_data_features_799>
<sub_sample_data_features_800>
<sub_sample_data_features_801>
<sub_sample_data_features_802>
<sub_sample_data_features_803>
<sub_sample_data_features_804>
<sub_sample_data_features_805>
<sub_sample_data_features_806>
<sub_sample_data_features_807>
<sub_sample_data_features_808>
<sub_sample_data_features_809>
<sub_sample_data_features_810>
<sub_sample_data_features_811>
<sub_sample_data_features_812>
<sub_sample_data_features_813>
<sub_sample_data_features_814>
<sub_sample_data_features_815>
<sub_sample_data_features_816>
<sub_sample_data_features_817>
<sub_sample_data_features_818>
<sub_sample_data_features_819>
<sub_sample_data_features_820>
<sub_sample_data_features_821>
<sub_sample_data_features_822>
<sub_sample_data_features_823>
<sub_sample_data_features_824>
<sub_sample_data_features_825>
<sub_sample_data_features_826>
<sub_sample_data_features_827>
<sub_sample_data_features_828>
<sub_sample_data_features_829>
<sub_sample_data_features_830>
<sub_sample_data_features_831>
<sub_sample_data_features_832>
<sub_sample_data_features_833>
<sub_sample_data_features_834>
<sub_sample_data_features_835>
<sub_sample_data_features_836>
<sub_sample_data_features_837>
<sub_sample_data_features_838>
<sub_sample_data_features_839>
<sub_sample_data_features_840>
<sub_sample_data_features_841>
<sub_sample_data_features_842>
<sub_sample_data_features_843>
<sub_sample_data_features_844>
<sub_sample_data_features_845>
<sub_sample_data_features_846>
<sub_sample_data_features_847>
<sub_sample_data_features_848>
<sub_sample_data_features_849>
<sub_sample_data_features_850>
<sub_sample_data_features_851>
<sub_sample_data_features_852>
<sub_sample_data_features_853>
<sub_sample_data_features_854>
<sub_sample_data_features_855>
<sub_sample_data_features_856>
<sub_sample_data_features_857>
<sub_sample_data_features_858>
<sub_sample_data_features_859>
<sub_sample_data_features_860>
<sub_sample_data_features_861>
<sub_sample_data_features_862>
<sub_sample_data_features_863>
<sub_sample_data_features_864>
<sub_sample_data_features_865>
<sub_sample_data_features_866>
<sub_sample_data_features_867>
<sub_sample_data_features_868>
<sub_sample_data_features_869>
<sub_sample_data_features_870>
<sub_sample_data_features_871>
<sub_sample_data_features_872>
<sub_sample_data_features_873>
<sub_sample_data_features_874>
<sub_sample_data_features_875>
<sub_sample_data_features_876>
<sub_sample_data_features_877>
<sub_sample_data_features_878>
<sub_sample_data_features_879>
<sub_sample_data_features_880>
<sub_sample_data_features_881>
<sub_sample_data_features_882>
<sub_sample_data_features_883>
<sub_sample_data_features_884>
<sub_sample_data_features_885>
<sub_sample_data_features_886>
<sub_sample_data_features_887>
<sub_sample_data_features_888>
<sub_sample_data_features_889>
<sub_sample_data_features_890>
<sub_sample_data_features_891>
<sub_sample_data_features_892>
<sub_sample_data_features_893>
<sub_sample_data_features_894>
<sub_sample_data_features_895>
<sub_sample_data_features_896>
<sub_sample_data_features_897>
<sub_sample_data_features_898>
<sub_sample_data_features_899>
<sub_sample_data_features_900>
<sub_sample_data_features_901>
<sub_sample_data_features_902>
<sub_sample_data_features_903>
<sub_sample_data_features_904>
<sub_sample_data_features_905>
<sub_sample_data_features_906>
<sub_sample_data_features_907>
<sub_sample_data_features_908>
<sub_sample_data_features_909>
<sub_sample_data_features_910>
<sub_sample_data_features_911>
<sub_sample_data_features_912>
<sub_sample_data_features_913>
<sub_sample_data_features_914>
<sub_sample_data_features_915>
<sub_sample_data_features_916>
<sub_sample_data_features_917>
<sub_sample_data_features_918>
<sub_sample_data_features_919>
<sub_sample_data_features_920>
<sub_sample_data_features_921>
<sub_sample_data_features_922>
<sub_sample_data_features_923>
<sub_sample_data_features_924>
<sub_sample_data_features_925>
<sub_sample_data_features_926>
<sub_sample_data_features_927>
<sub_sample_data_features_928>
<sub_sample_data_features_929>
<sub_sample_data_features_930>
<sub_sample_data_features_931>
<sub_sample_data_features_932>
<sub_sample_data_features_933>
<sub_sample_data_features_934>
<sub_sample_data_features_935>
<sub_sample_data_features_936>
<sub_sample_data_features_937>
<sub_sample_data_features_938>
<sub_sample_data_features_939>
<sub_sample_data_features_940>
<sub_sample_data_features_941>
<sub_sample_data_features_942>
<sub_sample_data_features_943>
<sub_sample_data_features_944>
<sub_sample_data_features_945>
<sub_sample_data_features_946>
<sub_sample_data_features_947>
<sub_sample_data_features_948>
<sub_sample_data_features_949>
<sub_sample_data_features_950>
<sub_sample_data_features_951>
<sub_sample_data_features_952>
<sub_sample_data_features_953>
<sub_sample_data_features_954>
<sub_sample_data_features_955>
<sub_sample_data_features_956>
<sub_sample_data_features_957>
<sub_sample_data_features_958>
<sub_sample_data_features_959>
<sub_sample_data_features_960>
<sub_sample_data_features_961>
<sub_sample_data_features_962>
<sub_sample_data_features_963>
<sub_sample_data_features_964>
<sub_sample_data_features_965>
<sub_sample_data_features_966>
<sub_sample_data_features_967>
<sub_sample_data_features_968>
<sub_sample_data_features_969>
<sub_sample_data_features_970>
<sub_sample_data_features_971>
<sub_sample_data_features_972>
<sub_sample_data_features_973>
<sub_sample_data_features_974>
<sub_sample_data_features_975>
<sub_sample_data_features_976>
<sub_sample_data_features_977>
<sub_sample_data_features_978>
<sub_sample_data_features_979>
<sub_sample_data_features_980>
<sub_sample_data_features_981>
<sub_sample_data_features_982>
<sub_sample_data_features_983>
<sub_sample_data_features_984>
<sub_sample_data_features_985>
<sub_sample_data_features_986>
<sub_sample_data_features_987>
<sub_sample_data_features_988>
<sub_sample_data_features_989>
<sub_sample_data_features_990>
<sub_sample_data_features_991>
<sub_sample_data_features_992>
<sub_sample_data_features_993>
<sub_sample_data_features_994>
<sub_sample_data_features_995>
<sub_sample_data_features_996>
<sub_sample_data_features_997>
<sub_sample_data_features_998>
<sub_sample_data_features_999>
<sub_sample_data_features_1000>
```

```
%matplotlib inline
import scipy
import numpy as np
import matplotlib.pyplot as plt
import numpy.linalg as linalg, impute_inv
%matplotlib widget
from ipywidgets import interact, interactive, fixed, interact_manual
import ipywidgets as widgets
from scipy.integrate import

# Some useful utilities
from mone_utils.ext_plot import matrix_hold,mv_hair, logscaleMpf, plot_bivariate_gauss, eval_func_on_grid, sub_sample_data

def compose(*functions):
    '''Compose a list of functions'
    return functools.reduce(lambda f, g: lambda x: f(g(x)), functions, lambda x: x)

def odeFunc(t,x):
    N = 100
    S = x[0]
    I = x[1]
    R = x[2]
    beta = 0.2
    gamma = 0.1
    delta = 0.15
    # theta = np.array([beta, gamma, delta])# true parameter settings
    # dS = -beta*S*I/N + delta*R # dS / d
    # dI = beta*S*I - gamma*I # dI / d
    # dR = gamma*I - delta*R # dR / d
    return [dS,dI,dR]

def solveFunc(t):
    timeTmp = np.linspace(0,6,6)
    N = 100
    I_0 = 10
    R_0 = 0
    ret = odeint(odeFunc, (N-I_0-R_0,I_0,R_0), t)
    soln = solp.integrate.solve_ivp(odeFunc,[0,6],[N-I_0-R_0,I_0],t_eval=timeTmp,method='RK45')
    return soln

soln1 = solveFunc(t)
data1 = np.concat(t)
stdNoise = 50
for i in range (1,len(data1)):
    data1[i]=soln1.y[1,i]+stdNoise*np.random.randn()

plt.figure(num='Indentifiable Version',figsize=(10,8))
plt.plot(soln1.t,soln1.y[0,:],'y',linewidth=4,label='Susceptible')
plt.plot(soln1.t,soln1.y[1,:],'r',linewidth=4,label='Infected')
plt.plot(soln1.t,soln1.y[2,:],'b',linewidth=4,label='Recovered')
plt.legend();
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()

plt.figure(num='Data',figsize=(10,8))
plt.plot(soln1.t,soln1.y[1,:],'r',linewidth=4,label='Infected')
plt.scatter(soln1.t,data1, s=7, c='r', marker='X',s=12,label='Data')
plt.legend();
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()

def odeFunc(t):
    timeTmp = np.linspace(0,6,6)
    N_0 = 500
    I_0 = 10
    R_0 = 0
    ret = np.array([N_0, I_0, R_0])
    soln = solp.integrate.solve_ivp(odeFunc,[0,6],[N_0,I_0],t_eval=timeTmp,method='RK45')
    return soln

soln1 = solveFunc(t)
data1 = np.concat(t,t)
stdNoise = 50
for i in range (0,len(data1)):
    data1[i]=soln1.y[1,i]+stdNoise*np.random.randn()

plt.figure(num='Indentifiable Version',figsize=(10,8))
plt.plot(soln1.t,soln1.y[0,:],'y',linewidth=4,label='Susceptible')
plt.plot(soln1.t,soln1.y[1,:],'r',linewidth=4,label='Infected')
plt.plot(soln1.t,soln1.y[2,:],'b',linewidth=4,label='Recovered')
plt.legend();
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()

plt.figure(num='Data',figsize=(10,8))
plt.plot(soln1.t,soln1.y[1,:],'r',linewidth=4,label='Infected')
plt.scatter(soln1.t,data1, s=7, c='r', marker='X',s=12,label='Data')
plt.legend();
plt.xlabel('Time')
plt.ylabel('Population')
plt.show()

beta = 0.2
r = 0.1
delta = 0.15
theta_t = np.array([beta, r, delta]) # true parameter settings
#theta = np.random.rand(3)
gamma = 1-r
gamma = 0.2
kappa = 0.1
theta = np.array([gamma, kappa,r, delta])# true parameter settings
print(theta_t0)


def log_likelihood(thetaarr,data):
    timeTmp = np.linspace(0,6,6)
    beta = theta[0]
    r = theta[1]
    delta = theta[2]
    N_0 = 500
    I_0 = 10
    R_0 = 0
    N_0 = np.array([N_0, I_0, R_0])
    soln = solp.integrate.solve_ivp(odeFunc,[0,6],[N_0],t_eval=timeTmp)

#     print(np.round(np.sum(soln.y,axis=0)))
#     print(np.round(np.sum(soln.y,axis=0)[-6:]))
#     return 1
    if theta[0]>0 and theta[1]>= 0 and theta[2]>= 0:
        sum = 0
        for i in range(1,len(data)):
            pm_exp_term = (-1)*np.sqrt(2*np.pi)*r*soln.y[1,i]
            sum = sum+(-1/(2*r**2)*(data[i]-soln.y[1,i])**2)+pm_exp_term
        if sum <=0:
            return sum
        else:
            print("Warning # frequency of occurrence updates (should be an integer > 0)
            if sd >= Nmax:
                self.sd = (2.4*r2) / self.dim
            else:
                self.sd = sd # scale for the covariance
        self.max_samples = max_samples # maximum number of samples
        self.max_iter = max_iter # maximum number of iterations
        self.num_samples = 0 # number of samples generated
        self.samples = np.zeros(max_samples, self.dim) # store the samples
        self.logpf_vals = np.zeros(max_samples)

        self.sample_mean = self.samples[0,:]

    def sample(self, initial_samples, num_samples):

        assert num_samples <= self.max_samples, "Requesting more samples than space is allocated for"

        self.samples[0, :] = initial_sample
        self.logpf_vals[0] = self.logpf(initial_sample)

        accept = 1
        for i in range(1, num_samples):

            # propose
            y = self.samples[i-1, :] + np.dot(self.cov_chol, np.random.randn(self.dim))
            y_logpf = self.logpf(y)

            # compute accept-reject probability, using the fact that we have a symmetric proposal
            a = np.exp(y_logpf - self.logpf_vals[i-1])
            a = min(a, 1)

            u = np.random.rand();

            if u < a:
                self.samples[i, :] = y
                self.logpf_vals[i] = y_logpf
                accept += 1
            else:
                # reject proposal
                self.num_samples += 1
                choldiag = np.diag(self.cov_chol) # sqrt of the covariance
                y1 = self.samples[i-1, :] + np.dot(choldiag, np.random.randn(self.dim))
                y_logpf1 = self.logpf(y1)

                x1y1 = np.exp(y_logpf - y_logpf1)
                x1y1 = np.exp(y_logpf - self.logpf_vals[i-1])

                q1 = multivariate_normal.pdf(y1,self.cov)
                q2 = multivariate_normal.pdf(self.samples[i-1, :],self.cov)

                a2 = np.exp(y_logpf2+np.log(q1)-x1y1)+(self.logpf_vals[i-1]+np.log(q2-x1y1) + q1 - q2)

                a2 = min(a2,1)

                u2 = np.random.rand()

                if u2 < a2:
                    self.samples[i, :] = y1
                    self.logpf_vals[i] = y_logpf2
                    accept += 1
                else:
                    self.samples[i, :] = self.samples[i-1, :]
                    self.logpf_vals[i] = self.logpf_vals[i-1]

        num_samples += 1

        # adapt covariance if its time
        if (i > self.dim and i % self.freq == 0:
            samplesMeanOver = i/(i/(i)).*self.samples[1, :]+(i/(i/(i))).*self.sample_mean
            cov0part = (i-1)/i*self.cov + (i-1)/(i-1)*np.dot(self.sample_mean.T,self.sample_mean) - \
                                    i/(i-1)*np.dot(samplesMeanOver,samplesMeanOver.T) + \
                                    np.dot(self.samples[i, :],self.samples[i, :])
            self.cov = (i-1)/i*cov0part + self.sd/i*np.eye(self.dim)
            self.cov_chol = np.linalg.cholesky(self.cov)
            self.sample_mean = samplesMeanOver
            if i % self.freq == 0:
                print("Finished sample {:d}, acceptance ratio = {accept / self.num_samples}")

        return self.samples, accept / float (self.num_samples)

def laplace_approx(initial_guess, logpost):
    '''Perform the laplace approximation,
        returning the MAP point and an approximation of the covariance

    Inputs
    ------
    initial_guess: (npars, ) array of initial parameters
    logpost: function (pars) -> log posterior

    Inputs
    ------
    map_point: (npars, ) MAP of the posterior
    cov_approx: (npars, npars), covariance matrix for Gaussian fit at MAP

    def neg_post(x):
        '''Negative posterior because optimizer is a minimizer'''
        return -logpost(x)

    # Gradient-free method to obtain optimum
    opt = scipy.optimize.minimize(neg_post, initial_guess, method='Nelder-Mead')
    # Gradient method will give approximation the inverse of the hessian
    res = scipy.optimize.minimize(neg_post, res.x)

    map_point = res.x
    cov_approx = res.hess_inv
    return map_point, cov_approx

num_samples = 3000
dims=2
queue = np.random.randn(num-dim) # random queue
map_point, cov_laplace = laplace_approx(queue, logpost0)
initial_sample = map_point
print(map_point)
print(cov_laplace)
print(map_point)

draw = DelayedMetJwithAmpliorationMetropolis(logpost1, cov=cov, freq=7, ub=200, sd = Nmax, eps = 1e-7, max_samples=num_samples)

samples, ar = draw.sample(initial_sample,num_samples)
print(samples)

def sub_sample_data(samples, frac_burnin=0.2, frac_use=0.7):
    '''Resample data by burning off the front fraction and using another fraction

    Inputs
    ------
    samples: (N, d) array of samples
    frac_burnin: fraction of the number of samples from the front to ignore
    frac_use: percentage of samples to use after burning, uniformly spaced

    nsamples = samples.shape[0]
    ind = np.arange(nsamples, dtype=np.int)
    start = int(frac_burn * nsamples)
    inds = inds[start:]
    nsamples = nsamples - start
    stride = int(1/frac_use * (nsamples * frac_use))
    inds = np.arange(start, nsamples, stride)
    return samples[inds, :]

samples = sub_sample_data(samples, frac_burn=0.2, frac_use=0.7)
fig, axs, gs = scatter_matrix(samples,nbh, bin_plot=False, gamma=0.2, labels=['$\theta_{0}$', '$\theta_{1}$', '$\theta_{2}$'])
fig.set_size_inches(16,16)
plt.subplots_adjust(left=0.01, right=0.99, wspace=0.2, hspace=0.2)

def autocorrelation(samples, maxlag=20, step=1):
    '''Compute the correlation of a set of samples

    Inputs
    ------
    samples: (N, d)
    maxlag: maximum distance to compute the correlation for
    step: step between distances from 1 to maxlag for which to compute teh correlations

    # Get the shapes
    nsim = samples.shape[1]
    nsamples = samples.shape[0]

    # Compute the mean
    mean = np.mean(samples, axis=0)

    # Compute the denominator, which is variance
    denominator = np.zeros(nsim)
    for i in range(nsim):
        denominator = denominator + (samples[:, i] - mean)**2

    lags = np.arange(0, maxlag, step)
    acorr = np.zeros(len(lags), nsim)
    for ix1 lag in enumerate(lags):
        # print("Computing lag {:d}".format(lag))
        # Compute the covariance between all samples *lag apart*
        for i in range(nsamples - lag):
            numerator = (samples[i, :] - mean)*(samples[i+ lag,:] -mean)
            acorr[ix1, :] = acorr[ix1, :]+numerator
        return lags, acorr

maxlag=500
step=1
lags, autoclag = autocorrelation(samples, maxlag=maxlag,step=step)
fig, axs = plt.subplots(3, figsize=(15,5))
axs[0].plot(lags, autoclag[:, 0], '.-')
axs[0].set_title('lag')
axs[0].set_ylabel('autocorrelation dimension 1')
axs[1].plot(lags, autoclag[:, 1], '.-')
axs[1].set_title('lag')
axs[1].set_ylabel('autocorrelation dimension 2')
axs[2].plot(lags, autoclag[:, 2], '.-')
axs[2].set_title('lag')
axs[2].set_ylabel('autocorrelation dimension 3')
plt.show()

ind0 = int(np.sum(autoclag[:,0]))
print(ind0)
ind1 = int(np.sum(autoclag[:,1]))
print(ind1)
ind2 = int(np.sum(autoclag[:,2]))
print(ind2)


fig, axs = plt.subplots(3, figsize=(15,5))
axs[0].plot(samples[:, 0], '.-')
axs[0].set_title('Sample Number')
axs[0].plot(samples[:, 1], '.-')
axs[1].set_title('Sample Number')
axs[1].plot(samples[:, 2], '.-')
axs[2].set_title('Sample Number')
axs[0].plot(samples[:, 0], '.-', fontsize=4)
axs[1].plot(samples[:, 1], '.-', fontsize=4)
axs[2].plot(samples[:, 2], '.-', fontsize=4)
axs[0].set_title('Sample Number', fontsize=4)
axs[1].set_title('Sample Number', fontsize=4)
axs[2].set_title('Sample Number', fontsize=4)
#axs[1].set_xlim(4000, 50000)


def log_likelihood(thetaarr,data):
    timeTmp = np.linspace(0,6,6)
    gamma = theta[0]
    kappa = theta[1]
    r = theta[2]
    delta = theta[3]
    N_0 = 500
    I_0 = 10
    R_0 = 0
    N = 100
    N_0 = np.array([N_0, I_0, R_0])
    soln = solp.integrate.solve_ivp(odeFunc,[0,6], gamma_0, method='RK45', t_eval=timeTmp)

#     print(np.round(np.sum(soln.y,axis=0)))
#     print(np.round(np.sum(soln.y,axis=0)[-6:]))
#     return -np.inf
    if theta[0]>0 and theta[1]>= 0 and theta[2]>= 0 and theta[3]>= 0:
        sum = 0
        pm_exp_term = (-1)*np.sqrt(2*np.pi)*r*
        exp_term = -1/2*np.dot(data[1:]-soln.y[1,1:],
        for i in range(1,len(soln.y[1,:])):
            f[i] = np.log(soln.y[1,i])
            f_theta1 = np.log(soln.y[1,i]*soln.y[0,i])
            f_theta2 = np.log(soln.y[1,i]*soln.y[2,i])
            f_theta3 = np.log(soln.y[1,i]*soln.y[1,i])
            f_theta = np.array([f_theta1, f_theta2, f_theta3])
            sum=sum.sum()+ f_theta

        else:
            return -np.inf

        return sum

logpost0 = lambda params: log_likelihood(params, data0)


def multVarGaussPDF(x,y,cov):
    x_m = x-y
    logpdf = np.log(((1/np.sqrt((2*np.pi)**2*np.linalg.det(cov)))*np.exp(-0.5*np.dot(np.dot(x_m).T,np.dot(np.linalg.inv(cov),x_m)))
    return logpdf

class DelayedMetJwithAmpliorationMetropolis:

    def __init__(self, logpdf, cov, freq=7, ub=200, sd=None, max_samples=10000, eps=1e-7):
        '''The class constructor, parameters are documented below'''
        # Initialize the sampler, keep the parameters we need defining the logpdf
        self.cov = cov # the initial covariance
        self.cov_chol = np.linalg.cholesky(cov) # sqrt of the covariance
        self.dim = self.cov.shape[0] # number of parameters
        self.ub = ub # time to start adapting
        self.freq = freq # frequency of covariance updates (should be an integer > 0)
        if sd == None:
            self.sd = (2.4*r2) / self.dim
        else:
            self.sd = sd # scale for the covariance
        self.max_samples = max_samples # maximum number of samples
        self.max_iter = max_iter # maximum number of iterations
        self.num_samples = 0 # number of samples generated
        self.samples = np.zeros(max_samples, self.dim) # store the samples
        self.logpf_vals = np.zeros(max_samples)

        self.sample_mean = self.samples[0,:]

    def sample(self, initial_samples, num_samples):

        assert num_samples <= self.max_samples, "Requesting more samples than space is allocated for"

        self.samples[0, :] = initial_sample
        self.logpf_vals[0] = self.logpf(initial_sample)

        accept = 1
        for i in range(1, num_samples):

            # propose
            y = self.samples[i-1, :] + np.dot(self.cov_chol, np.random.randn(self.dim))
            y_logpf = self.logpf(y)

            # compute accept-reject probability, using the fact that we have a symmetric proposal
            a = np.exp(y_logpf - self.logpf_vals[i-1])
            a = min(a, 1)

            u = np.random.rand();

            if u < a:# accept
                self.samples[i, :] = y
                self.logpf_vals[i] = y_logpf
                accept += 1
            else:
                self.samples[i, :] = self.samples[i-1, :]
                self.logpf_vals[i] = self.logpf_vals[i-1]

        num_samples += 1

        # adapt covariance if its time
        if (i > self.dim and i % self.freq == 0:
            samplesMeanOver = i/(i/(i))*self.samples[1, :]+(i/(i/(i)))*self.sample_mean
            cov0part = (i-1)/i*self.cov + (i-1)/(i-1)*np.dot(self.sample_mean.T,self.sample_mean) - \
                                    i/(i-1)*np.dot(samplesMeanOver,samplesMeanOver.T) + \
                                    np.dot(self.samples[i, :],self.samples[i, :])
            self.cov = (i-1)/i*cov0part + self.sd/i*np.eye(self.dim)
            self.cov_chol = np.linalg.cholesky(self.cov)
            self.sample_mean = samplesMeanOver
            if i % self.freq == 0:
                print("Finished sample {:d}, acceptance ratio = {accept / self.num_samples}")

        return self.samples, accept / float (self.num_samples)

def laplace_approx(initial_guess, logpost):
    '''Perform the laplace approximation,
        returning the MAP point and an approximation of the covariance

    Inputs
    ------
    initial_guess: (npars, ) array of initial parameters
    logpost: function (pars) -> log posterior

    Inputs
    ------
    map_point: (npars, ) MAP of the posterior
    cov_approx: (npars, npars), covariance matrix for Gaussian fit at MAP

    def neg_post(x):
        '''Negative posterior because optimizer is a minimizer'''
        return -logpost(x)

    # Gradient-free method to obtain optimum
    opt = scipy.optimize.minimize(neg_post, initial_guess, method='Nelder-Mead')
    # Gradient method will give approximation the inverse of the hessian
    res = scipy.optimize.minimize(neg_post, res.x)

    map_point = res.x
    cov_approx = res.hess_inv
    return map_point, cov_approx

num_samples = 3000
dims=4
queue = np.random.randn(num-dim) # random queue
map_point, cov_laplace = laplace_approx(queue, logpost0)
initial_sample = map_point
print(map_point)
print(cov_laplace)
print(map_point)

draw = DelayedMetJwithAmpliorationMetropolis(logpost1, cov=cov, freq=7, ub=200, sd = Nmax, eps = 1e-7, max_samples=num_samples)

samples, ar = draw.sample(theta_t0,num_samples)
print(samples)

samples = sub_sample_data(samples, frac_burnin=0.2, frac_use=0.7)
fig, axs, gs = scatter_matrix(samples,nbh, bin_plot=False, gamma=0.2)
fig.set_size_inches(16,16)
plt.subplots_adjust(left=0.01, right=0.99, wspace=0.2, hspace=0.2)

maxlag=500
step=1
lags, autoclag = autocorrelation(samples, maxlag=maxlag,step=step)
fig, axs = plt.subplots(4, figsize=(15,5))
axs[0].plot(lags, autoclag[:, 0], '.-')
axs[0].set_title('lag')
axs[0].set_ylabel('autocorrelation dimension 1')
axs[1].plot(lags, autoclag[:, 1], '.-')
axs[1].set_title('lag')
axs[1].set_ylabel('autocorrelation dimension 2')
axs[2].plot(lags, autoclag[:, 2], '.-')
axs[2].set_title('lag')
axs[2].set_ylabel('autocorrelation dimension 3')
axs[3].plot(lags, autoclag[:, 3], '.-')
axs[3].set_title('lag')
axs[3].set_ylabel('autocorrelation dimension 4')
plt.show()

ind0 = int(np.sum(autoclag[:,0]))
print(ind0)
ind1 = int(np.sum(autoclag[:,1]))
print(ind1)
ind2 = int(np.sum(autoclag[:,2]))
print(ind2)
ind3 = int(np.sum(autoclag[:,3]))
print(ind3)


fig, axs = plt.subplots(3, figsize=(15,10))
axs[0].plot(samples[:, 0], '.-')
axs[0].set_title('Sample Number')
axs[0].plot(samples[:, 1], '.-')
axs[1].set_title('Sample Number')
axs[1].plot(samples[:, 2], '.-')
axs[2].set_title('Sample Number')
axs[0].plot(samples[:, 0], '.-', fontsize=4)
axs[1].plot(samples[:, 1], '.-', fontsize=4)
axs[2].plot(samples[:, 2], '.-', fontsize=4)
axs[0].set_title('Sample Number', fontsize=4)
axs[1].set_title('Sample Number', fontsize=4)
axs[2].set_title('Sample Number', fontsize=4)
#axs[1].set_xlim(4000, 50000)
```